شرح سطر بسطر ٤ — المكوّنات المشتركة (core/widgets)

مشروع رِزمة · Flutter · كل سطر مع سبب كتابته بهذه الطريقة

هذه ودجتات تتكرر في أكثر من شاشة، فكتبناها مرة واحدة: الزر الرئيسي، مربع الأيقونة، شريط التقدّم، الشريط السفلي، الشخصية الملوّنة، الشريط العلوي وزر الرجوع، المفتاح، البطاقة، الحد المنقّط، ونقاط/لوحة الرمز السري.

القاعدة العامة: كل ودجت بلا حالة داخلية (`StatelessWidget`) ويستقبل ما يحتاجه كمعاملات، فيسهل اختباره وإعادة استعماله.

تذكير بالاتجاه: في RTL يظهر أول عنصر في `Row` على اليمين؛ لذلك ترتيب العناصر في الكود هو ترتيب القراءة العربي.

lib/core/widgets/primary_button.dart — الزر الرئيسي

lib/core/widgets/primary_button.dart

الأسطر 1–5

```
1  import 'package:flutter/material.dart';
2  import 'package:flutter_svg/flutter_svg.dart';
3
4  import '../theme/app_colors.dart';
5  import '../theme/app_text_styles.dart';
```

ماذا يفعل؟ استيراد Material وSVG والألوان وأنماط النص.

لماذا بهذه الطريقة؟ ما يحتاجه الزر لرسم نفسه.

الأسطر 7–8

```
7  /// الزر الرئيسي العريض. إذا كان [onPressed] فارغًا يظهر معطّلًا.
8  class PrimaryButton extends StatelessWidget {
```

ماذا يفعل؟ تعليق وصنف `PrimaryButton` بلا حالة.

لماذا بهذه الطريقة؟ الحالة (مفعّل/معطّل) تأتي من الخارج عبر `onPressed`.

الأسطر 9–17

```
9    const PrimaryButton({
10     super.key,
11     required this.label,
12     required this.onPressed,
13     this.iconAsset,
14     this.height = 52,
15     this.backgroundColor = AppColors.teal,
16     this.foregroundColor = AppColors.card,
17   });
```

ماذا يفعل؟ الباني: `label` و`onPressed` مطلوبان، والباقي اختياري بقيم افتراضية من التصميم (ارتفاع 52، خلفية teal، نص أبيض).

لماذا بهذه الطريقة؟ تمرير `onPressed: null` معناه «معطّل»، وهذا العرف المعتاد في Flutter.

الأسطر 19–24

```
19    final String label;
20    final VoidCallback? onPressed;
21    final String? iconAsset;
22    final double height;
23    final Color backgroundColor;
24    final Color foregroundColor;
```

ماذا يفعل؟ الحقول.

لماذا بهذه الطريقة؟ `VoidCallback?` تسمح بحالة التعطيل. الأيقونة اختيارية لأن بعض الأزرار بلا أيقونة.

الأسطر 26–28

```
26    @override
27    Widget build(BuildContext context) {
28      final enabled = onPressed != null;
```

ماذا يفعل؟ نحسب `enabled` من كون `onPressed` غير فارغ.

لماذا بهذه الطريقة؟ مصدر واحد للحقيقة للون وللوصول.

الأسطر 30–34

```
30      return Semantics(
31        button: true,
32        enabled: enabled,
33        label: label,
34        excludeSemantics: true,
```

ماذا يفعل؟ `Semantics`: زر، مفعّل أو لا، اسمه `label`، ونتجاهل الودجتات الداخلية.

لماذا بهذه الطريقة؟ قارئ الشاشة يقول «ابدأ، زر» مرة واحدة بدل قراءة النص والأيقونة منفصلين.

الأسطر 35–38

```
35        child: Material(
36          color: enabled ? backgroundColor : AppColors.disabled,
37          borderRadius: BorderRadius.circular(16),
38          clipBehavior: Clip.antiAlias,
```

ماذا يفعل؟ `Material` بلون الخلفية (رمادي إن كان معطّلًا) وزوايا 16 وقصّ.

لماذا بهذه الطريقة؟ يحتاج `InkWell` إلى `Material` ليرسم التموّج. `Clip.antiAlias` يمنع خروج التموّج عن الزوايا المدوّرة.

الأسطر 39–40

```
39          child: InkWell(
40            onTap: onPressed,
```

ماذا يفعل؟ `InkWell` ينفّذ `onPressed` عند الضغط.

لماذا بهذه الطريقة؟ عندما تكون `null` يتعطل اللمس تلقائيًا.

الأسطر 41–44

```
            child: Container(
              height: height,
              padding: const EdgeInsets.symmetric(horizontal: 20),
              alignment: Alignment.center,
```

ماذا يفعل؟ `Container` بارتفاع `height` وحشوة أفقية 20 ومحاذاة للمنتصف.

لماذا بهذه الطريقة؟ الارتفاع ثابت كما في Figma مهما كان طول النص.

الأسطر 45–48

```
              child: Row(
                mainAxisSize: MainAxisSize.min,
                spacing: 8,
                children: [
```

ماذا يفعل؟ `Row` بحجم أدنى ومسافة 8 بين الأيقونة والنص.

لماذا بهذه الطريقة؟ `MainAxisSize.min` ليحيط الصف بمحتواه فيتوسّط. خاصية `spacing` تغني عن `SizedBox` بين العناصر.

الأسطر 49–50

```
                  if (iconAsset != null)
                    SvgPicture.asset(iconAsset!, width: 20, height: 20),
```

ماذا يفعل؟ إن وُجدت أيقونة تُرسم بحجم 20 وهي أول عنصر.

لماذا بهذه الطريقة؟ أول عنصر في RTL = على اليمين من النص، كما في Figma.

الأسطر 51–56

```
                  Text(
                    label,
                    style: AppTextStyles.button16.copyWith(
                      color: foregroundColor,
                    ),
                  ),
```

ماذا يفعل؟ نص الزر بنمط `button16` مع لون `foregroundColor`.

لماذا بهذه الطريقة؟ `copyWith` يغيّر اللون فقط (أبيض على الأخضر، أخضر على الأبيض).

الأسطر 57–64

```
                ],
              ),
            ),
          ),
        ),
      );
    }
  }
```

ماذا يفعل؟ إغلاق الأقواس.

لماذا بهذه الطريقة؟ نهاية الصنف.

lib/core/widgets/icon_tile.dart — مربع الأيقونة الملوّن

lib/core/widgets/icon_tile.dart

الأسطر 1–2

```dart
import 'package:flutter/material.dart';
import 'package:flutter_svg/flutter_svg.dart';
```

ماذا يفعل؟ استيراد Material وSVG.

لماذا بهذه الطريقة؟ الأيقونة SVG أو Material.

الأسطر 4–7

```dart
/// مربع ملوّن بزوايا مدوّرة يحوي أيقونة في الوسط.
/// الأيقونة إما ملف SVG من Figma ([asset])، أو أيقونة Material ([iconData])
/// للأيقونات التي لم تُصدَّر من Figma بعد.
class IconTile extends StatelessWidget {
```

ماذا يفعل؟ تعليق وصنف `IconTile`.

لماذا بهذه الطريقة؟ الأيقونة إما `asset` (SVG من Figma) أو (Material) `iconData` لما لم يُصدَّر بعد.

الأسطر 8–16

```dart
  const IconTile({
    super.key,
    this.asset,
    this.iconData,
    this.iconColor,
    required this.color,
    this.size = 48,
    this.iconSize = 24,
    this.radius = 14,
```

ماذا يفعل؟ الباني: كل شيء اختياري عدا `color`.

لماذا بهذه الطريقة؟ القيم الافتراضية الأكثر استعمالًا في التصميم (48 / 24 / زوايا 14).

الأسطر 17–20

```dart
  }) : assert(
         (asset != null) != (iconData != null),
         'مرّر asset أو iconData (واحدًا فقط)',
       );
```

ماذا يفعل؟ `assert` يتأكد أنه مُرّر واحد فقط من `asset` و`iconData`.

لماذا بهذه الطريقة؟ يلتقط الخطأ أثناء التطوير (الاثنان معًا أو لا شيء).

الأسطر 22–28

```
22    final String? asset;
23    final IconData? iconData;
24    final Color? iconColor;
25    final Color color;
26    final double size;
27    final double iconSize;
28    final double radius;
```

ماذا يفعل؟ الحقول.

لماذا بهذه الطريقة؟ كلها `final` فالودجت غير قابل للتغيير.

الأسطر 30–31

```
30    @override
31    Widget build(BuildContext context) {
```

ماذا يفعل؟ دالة `build`.

لماذا بهذه الطريقة؟ تعيد شجرة المربع.

الأسطر 32–39

```
32      return Container(
33        width: size,
34        height: size,
35        decoration: BoxDecoration(
36          color: color,
37          borderRadius: BorderRadius.circular(radius),
38        ),
39        alignment: Alignment.center,
```

ماذا يفعل؟ `Container` بعرض وارتفاع `size` وخلفية ملوّنة وزوايا `radius` ومحاذاة للمنتصف.

لماذا بهذه الطريقة؟ محاذاة المنتصف تضع الأيقونة وسط المربع.

الأسطر 40–42

```
40        child: asset != null
41            ? SvgPicture.asset(asset!, width: iconSize, height: iconSize)
42            : Icon(iconData, size: iconSize, color: iconColor),
```

ماذا يفعل؟ إن كان `asset` نرسم SVG بحجم `iconSize`، وإلا أيقونة Material بلونها.

لماذا بهذه الطريقة؟ حجم الأيقونة يُحدَّد صراحةً كما يشترط تصدير Figma.

الأسطر 43–45

```
43      );
44    }
45  }
```

ماذا يفعل؟ إغلاق الأقواس.

لماذا بهذه الطريقة؟ نهاية الصنف.

lib/core/widgets/progress_bar.dart — شريط التقدّم

lib/core/widgets/progress_bar.dart

الأسطر 1–4

```
import 'package:flutter/material.dart';

/// شريط تقدّم يبدأ من اليمين (اتجاه القراءة العربي).
class ProgressBar extends StatelessWidget {
```

ماذا يفعل؟ استيراد Material وتعليق وصنف `ProgressBar`.

لماذا بهذه الطريقة؟ يبدأ من اليمين (اتجاه القراءة العربي).

الأسطر 5–11

```
  const ProgressBar({
    super.key,
    required this.value,
    required this.trackColor,
    required this.fillColor,
    this.height = 6,
  }) : assert(value >= 0 && value <= 1);
```

ماذا يفعل؟ الباني: `value` ولونا المسار والتعبئة مطلوبة، و`height` افتراضيه 6. و`assert` أن القيمة بين 0 و1.

لماذا بهذه الطريقة؟ يلتقط الخطأ في التطوير (مثل تمرير 75 بدل 0.75). التصميم يستعمل نفس الشريط بارتفاعين وألوان.

الأسطر 13–17

```
  /// بين 0 و 1.
  final double value;
  final Color trackColor;
  final Color fillColor;
  final double height;
```

ماذا يفعل؟ الحقول.

لماذا بهذه الطريقة؟ `value` بين 0 و1 ليتوافق مع `widthFactor` مباشرة.

الأسطر 19–21

```
  @override
  Widget build(BuildContext context) {
    final radius = BorderRadius.circular(height / 2);
```

ماذا يفعل؟ `build` ويحسب نصف قطر الزوايا = نصف الارتفاع.

لماذا بهذه الطريقة؟ يعطي أطرافًا دائرية تمامًا (شكل كبسولة) بأي ارتفاع.

الأسطر 23–26

```
23      return Container(
24        height: height,
25        decoration: BoxDecoration(color: trackColor, borderRadius: radius),
26        alignment: AlignmentDirectional.centerStart,
```

ماذا يفعل؟ `Container` للمسار: ارتفاع ولون ونصف قطر، ومحاذاة `AlignmentDirectional.centerStart`.

لماذا بهذه الطريقة؟ أهم سطر: `centerStart` = اليمين في RTL فتبدأ التعبئة من اليمين كما في `centerLeft`. Figma كان سيعكس الشكل.

الأسطر 27–33

```
27        child: FractionallySizedBox(
28          widthFactor: value,
29          heightFactor: 1,
30          child: DecoratedBox(
31            decoration: BoxDecoration(color: fillColor, borderRadius: radius),
32          ),
33        ),
```

ماذا يفعل؟ `FractionallySizedBox` بعرض `value` من عرض المسار، فيه مستطيل بلون التعبئة.

لماذا بهذه الطريقة؟ يحسب العرض نسبيًا تلقائيًا بلا `LayoutBuilder` ولا معرفة العرض بالبكسل.

الأسطر 34–36

```
34      );
35    }
36  }
```

ماذا يفعل؟ إغلاق الأقواس.

لماذا بهذه الطريقة؟ نهاية الصنف.

lib/core/widgets/bottom_nav.dart — الشريط السفلي

lib/core/widgets/bottom_nav.dart

الأسطر 1–7

```
1   import 'package:flutter/material.dart';
2   import 'package:flutter_svg/flutter_svg.dart';
3
4   import '../constants/app_assets.dart';
5   import '../strings/ar.dart';
6   import '../theme/app_colors.dart';
7   import '../theme/app_text_styles.dart';
```

ماذا يفعل؟ استيراد Material وSVG والأصول والنصوص والألوان والأنماط.

لماذا بهذه الطريقة؟ ما يحتاجه الشريط.

السطر 9

```
enum RizmaTab { home, subjects, progress, profile }
```

ماذا يفعل؟ تعداد `RizmaTab` بأربع قيم.

لماذا بهذه الطريقة؟ نمرّر التبويب الحالي قيمةً نوعية آمنة بدل رقم فيستحيل خلط ترتيبها.

الأسطر 11–13

```
/// الشريط السفلي: الرئيسية · المواد · تقدّمي · حسابي (من اليمين لليسار).
class BottomNav extends StatelessWidget {
  const BottomNav({super.key, required this.current, required this.onSelected});
```

ماذا يفعل؟ تعليق وصنف `BottomNav` والباني بحقلين مطلوبين.

لماذا بهذه الطريقة؟ الأولى يمينًا: الرئيسية ثم المواد ثم تقدّمي ثم حسابي.

الأسطر 15–16

```
  final RizmaTab current;
  final ValueChanged<RizmaTab> onSelected;
```

ماذا يفعل؟ الحقلان: التبويب الحالي ودالة الاختيار.

لماذا بهذه الطريقة؟ الشريط لا يحتفظ بحالة؛ من يستعمله يقرر الحالي ويمرّره.

الأسطر 18–35

```
  static const _items = [
    (tab: RizmaTab.home, label: ArStrings.navHome, asset: AppAssets.navHome),
    (
      tab: RizmaTab.subjects,
      label: ArStrings.navSubjects,
      asset: AppAssets.navBook,
    ),
    (
      tab: RizmaTab.progress,
      label: ArStrings.navProgress,
      asset: AppAssets.navChart,
    ),
    (
      tab: RizmaTab.profile,
      label: ArStrings.navProfile,
      asset: AppAssets.navUser,
    ),
  ];
```

ماذا يفعل؟ قائمة ثابتة من أربعة سجلات (record): التبويب والاسم والأيقونة.

لماذا بهذه الطريقة؟ سجلات Dart 3 المسمّاة أخف من صنف جديد لبيانات صغيرة ومحلية. ترتيب القائمة هو ترتيب Figma.

الأسطر 37–38

```
37    @override
38    Widget build(BuildContext context) {
```

ماذا يفعل؟ `build` .

لماذا بهذه الطريقة؟ يعيد الشريط.

الأسطر 39–43

```
39      return DecoratedBox(
40        decoration: const BoxDecoration(
41          color: AppColors.card,
42          border: Border(top: BorderSide(color: AppColors.line)),
43        ),
```

ماذا يفعل؟ `DecoratedBox` بخلفية بيضاء وحدّ علوي رفيع.

لماذا بهذه الطريقة؟ الحد يفصل الشريط عن المحتوى كما في Figma.

الأسطر 44–45

```
44        child: SafeArea(
45          top: false,
```

ماذا يفعل؟ `SafeArea` من الأسفل فقط.

لماذا بهذه الطريقة؟ تحمي الشريط من شريط الإيماءات أسفل الهاتف.

الأسطر 46–47

```
46          child: Padding(
47            padding: const EdgeInsetsDirectional.fromSTEB(16, 10, 16, 18),
```

ماذا يفعل؟ حشوة 16 أفقيًا و10 أعلى و18 أسفل.

لماذا بهذه الطريقة؟ قيم Figma (بصيغة `Directional`).

الأسطر 48–50

```
48            child: Row(
49              mainAxisAlignment: MainAxisAlignment.spaceBetween,
50              children: [
```

ماذا يفعل؟ `Row` بتوزيع `spaceBetween` .

لماذا بهذه الطريقة؟ يوزّع التبويبات الأربعة بالتساوي بين الحافتين.

الأسطر 51–60

```
51                // Flexible: على شاشة ضيقة أو بخط مكبّر تنكمش العناصر بدل أن تفيض.
52                for (final item in _items)
53                  Flexible(
54                    child: _NavItem(
55                      label: item.label,
56                      asset: item.asset,
57                      selected: item.tab == current,
58                      onTap: () => onSelected(item.tab),
59                    ),
60                  ),
```

ماذا يفعل؟ تعليق ثم حلقة تبني `_NavItem` لكل عنصر داخل `Flexible`؛ المحدَّد هو من يساوي `current`، والضغط يستدعي `onSelected`.

لماذا بهذه الطريقة؟ `Flexible` يجعل العنصر ينكمش عند الضيق: على شاشة 320 وخط مكبّر ×1.5 كان الصف يفيض بـ 63 بكسلًا (اكتشفه اختبار `layout_stress_test`). مع `spaceBetween` يبقى التوزيع نفسه على الشاشات العادية.

الأسطر 61–67

```
61              ],
62            ),
63          ),
64        ),
65      );
66    }
67  }
```

ماذا يفعل؟ إغلاق الأقواس.

لماذا بهذه الطريقة؟ نهاية `BottomNav`.

الأسطر 69–75

```
69  class _NavItem extends StatelessWidget {
70    const _NavItem({
71      required this.label,
72      required this.asset,
73      required this.selected,
74      required this.onTap,
75    });
```

ماذا يفعل؟ ودجت `_NavItem` خاص وباني بأربعة معاملات.

لماذا بهذه الطريقة؟ عنصر واحد من الشريط.

الأسطر 77–80

```
77    final String label;
78    final String asset;
79    final bool selected;
80    final VoidCallback onTap;
```

ماذا يفعل؟ الحقول.

لماذا بهذه الطريقة؟ غير قابلة للتغيير.

الأسطر 82–84

```
82    @override
83    Widget build(BuildContext context) {
84      final color = selected ? AppColors.teal : AppColors.soft;
```

ماذا يفعل؟ `build` ويحدد اللون: teal للمحدَّد وإلا `soft`.

لماذا بهذه الطريقة؟ لون الأيقونة والنص من متغيّر واحد فلا يختلفان.

الأسطر 86–90

```
86      return Semantics(
87        button: true,
88        selected: selected,
89        label: label,
90        excludeSemantics: true,
```

ماذا يفعل؟ `Semantics`: زر ومحدَّد أم لا واسمه.

لماذا بهذه الطريقة؟ قارئ الشاشة يقول «الرئيسية، محدَّد».

الأسطر 91–94

```
91        child: Material(
92          color: selected ? AppColors.tealTint : Colors.transparent,
93          borderRadius: BorderRadius.circular(16),
94          clipBehavior: Clip.antiAlias,
```

ماذا يفعل؟ `Material` بخلفية `tealTint` للمحدَّد وشفافة لغيره، وزوايا 16 وقصّ.

لماذا بهذه الطريقة؟ هذه «الكبسولة الخضراء» مؤشر التبويب الحالي في Figma.

الأسطر 95–96

```
95          child: InkWell(
96            onTap: onTap,
```

ماذا يفعل؟ `InkWell` ينفّذ `onTap`.

لماذا بهذه الطريقة؟ الضغط على أي مكان داخل العنصر.

الأسطر 97–98

```
97            child: Padding(
98              padding: const EdgeInsets.symmetric(horizontal: 12, vertical: 6),
```

ماذا يفعل؟ حشوة 12 أفقيًا و6 عموديًا.

لماذا بهذه الطريقة؟ من Figma.

الأسطر 99–102

```
            child: Column(
              mainAxisSize: MainAxisSize.min,
              spacing: 2,
              children: [
```

ماذا يفعل؟ `Column` بحجم أدنى ومسافة 2: أيقونة فوق النص.

لماذا بهذه الطريقة؟ `min` ليحيط العنصر بمحتواه فقط.

الأسطر 103–108

```
                SvgPicture.asset(
                  asset,
                  width: 22,
                  height: 22,
                  colorFilter: ColorFilter.mode(color, BlendMode.srcIn),
                ),
```

ماذا يفعل؟ الأيقونة بحجم 22 ومرشّح لوني `ColorFilter.mode(color, srcIn)`.

لماذا بهذه الطريقة؟ أيقونات التنقّل خطوط بلون واحد؛ `srcIn` يصبغها بلون التبويب فنستعمل ملفًّا واحدًا لكل أيقونة بدل ملفين.

الأسطر 109–116

```
                FittedBox(
                  fit: BoxFit.scaleDown,
                  child: Text(
                    label,
                    maxLines: 1,
                    style: AppTextStyles.captionBold13.copyWith(color: color),
                  ),
                ),
```

ماذا يفعل؟ نص الاسم بنمط `captionBold13` ولون التبويب، داخل `FittedBox(scaleDown)` وسطر واحد.

لماذا بهذه الطريقة؟ وزن 700 ليبرز بحجم صغير. `FittedBox` يصغّر النص إن لم يتسع بدل قصّه أو الفيض، ولا يغيّره في الحالة العادية.

الأسطر 117–124

```
              ],
            ),
          ),
        ),
      ),
    );
  }
}
```

ماذا يفعل؟ إغلاق الأقواس.

لماذا بهذه الطريقة؟ نهاية الملف.

lib/core/widgets/mascot.dart — الشخصية الملوّنة

lib/core/widgets/mascot.dart

الأسطر 1–6

```dart
import 'package:flutter/material.dart';
import 'package:flutter_svg/flutter_svg.dart';

import '../constants/app_assets.dart';
import '../db/enums.dart';
import '../theme/profile_palette.dart';
```

ماذا يفعل؟ استيراد Material وSVG والأصول والتعداد والألوان.

لماذا بهذه الطريقة؟ نحتاج `ColorMapper` من `flutter_svg`.

الأسطر 8–10

```dart
/// شخصية رِزمة بلون الطالب. الأصل أخضر، فنستبدل لونيه (الجسم والزاوية)
/// وقت العرض بدل الاحتفاظ بخمسة ملفات.
class Mascot extends StatelessWidget {
```

ماذا يفعل؟ تعليق وصنف `Mascot`.

لماذا بهذه الطريقة؟ الأصل أخضر؛ نستبدل لونيه وقت العرض بدل الاحتفاظ بخمسة ملفات.

الأسطر 11–14

```dart
  const Mascot({super.key, required this.size, this.color = ProfileColor.teal});

  final double size;
  final ProfileColor color;
```

ماذا يفعل؟ الباني: الحجم مطلوب واللون الافتراضي teal، والحقلان.

لماذا بهذه الطريقة؟ الحجم يتغيّر بحسب المكان (40 و56 و84 و96...).

الأسطر 16–25

```dart
  @override
  Widget build(BuildContext context) {
    return SvgPicture.asset(
      AppAssets.mascot150,
      width: size,
      height: size,
      colorMapper: _MascotColorMapper(body: color.body, fold: color.fold),
    );
  }
}
```

ماذا يفعل؟ `build`: يرسم `mascot_150.svg` بالحجم المطلوب ومعه مبدّل ألوان.

لماذا بهذه الطريقة؟ ملف واحد يتحمّل كل الأحجام لأنه متجه (SVG).

الأسطر 27–28

```
class _MascotColorMapper extends ColorMapper {
  const _MascotColorMapper({required this.body, required this.fold});
```

ماذا يفعل؟ صنف خاص `_MascotColorMapper` يرث `ColorMapper` وباني ثابت.

لماذا بهذه الطريقة؟ `ColorMapper` يتيح استبدال ألوان محدّدة وقت قراءة SVG.

الأسطر 30–31

```
  static const _originalBody = Color(0xFF177A63);
  static const _originalFold = Color(0xFF7ED3B2);
```

ماذا يفعل؟ لونا الشخصية الأصليان: الجسم `#177A63` والزاوية المطوية `#7ED3B2` .

لماذا بهذه الطريقة؟ هذان اللونان فقط يتغيّران؛ العيون والخدود والفم تبقى كما هي.

الأسطر 33–34

```
  final Color body;
  final Color fold;
```

ماذا يفعل؟ اللونان الجديدان.

لماذا بهذه الطريقة؟ من `ProfileColorPalette` .

الأسطر 36–46

```
  @override
  Color substitute(
    String? id,
    String elementName,
    String attributeName,
    Color color,
  ) {
    if (color == _originalBody) return body;
    if (color == _originalFold) return fold;
    return color;
  }
```

ماذا يفعل؟ `substitute` : إن كان اللون الأصلي هو لون الجسم نعيد الجديد، وكذلك الزاوية، وإلا نعيده كما هو.

لماذا بهذه الطريقة؟ تُستدعى لكل لون في ملف SVG.

الأسطر 48–53

```
  @override
  bool operator ==(Object other) =>
      other is _MascotColorMapper && other.body == body && other.fold == fold;

  @override
  int get hashCode => Object.hash(body, fold);
```

ماذا يفعل؟ تعريف `==` و`hashCode` على اللونين.

لماذا بهذه الطريقة؟ تخزّن flutter_svg الصور المحمَّلة مؤقتًا بمفتاح يشمل المبدِّل؛ بدون المساواة قد لا تُعاد الاستفادة منها.

السطر 54

```
}
```

ماذا يفعل؟ إغلاق الصنف.

لماذا بهذه الطريقة؟ نهاية الملف.

lib/core/widgets/screen_top_bar.dart — زر الرجوع والشريط العلوي

lib/core/widgets/screen_top_bar.dart

الأسطر 1–7

```
import 'package:flutter/material.dart';
import 'package:flutter_svg/flutter_svg.dart';

import '../constants/app_assets.dart';
import '../strings/ar.dart';
import '../theme/app_colors.dart';
import '../theme/app_text_styles.dart';
```

ماذا يفعل؟ استيراد Material وSVG والأصول والنصوص والألوان والأنماط.

لماذا بهذه الطريقة؟ ما يحتاجه الشريط.

الأسطر 9–13

```
/// زر الرجوع المربّع. أيقونته مرسومة بسهم لليمين = اتجاه الرجوع في RTL.
class BackTile extends StatelessWidget {
  const BackTile({super.key, required this.onTap});

  final VoidCallback onTap;
```

ماذا يفعل؟ تعليق وصنف `BackTile` ومعامل `onTap` .

لماذا بهذه الطريقة؟ أيقونة الرجوع سهم لليمين = اتجاه الرجوع في RTL.

الأسطر 15–20

```
  @override
  Widget build(BuildContext context) {
    return Semantics(
      button: true,
      label: ArStrings.back,
      excludeSemantics: true,
```

ماذا يفعل؟ `build` و`Semantics` باسم «رجوع».

لماذا بهذه الطريقة؟ الزر أيقونة بلا نص، فبدون الوصف لا يعرف مستخدم قارئ الشاشة وظيفته.

الأسطر 21–24

```
        child: Material(
          color: AppColors.mute,
          borderRadius: BorderRadius.circular(14),
          clipBehavior: Clip.antiAlias,
```

ماذا يفعل؟ `Material` بلون `mute` وزوايا 14 وقصّ.

لماذا بهذه الطريقة؟ نفس سبب باقي الأزرار (التموّج).

الأسطر 25–26

```
          child: InkWell(
            onTap: onTap,
```

ماذا يفعل؟ `InkWell` ينفّذ `onTap`.

لماذا بهذه الطريقة؟ الإجراء يأتي من الخارج.

الأسطر 27–34

```
            child: SizedBox(
              width: 44,
              height: 44,
              child: Center(
                child: SvgPicture.asset(AppAssets.back, width: 22, height: 22),
              ),
            ),
          ),
```

ماذا يفعل؟ مربع 44×44 فيه السهم 22 في المنتصف.

لماذا بهذه الطريقة؟ 44 حجم لمس مريح؛ الأيقونة جاهزة بالاتجاه الصحيح فلا نقلبها برمجيًا.

الأسطر 35–38

```
        ),
      );
    }
  }
```

ماذا يفعل؟ إغلاق الأقواس.

لماذا بهذه الطريقة؟ نهاية `BackTile`.

الأسطر 40–48

```
/// الشريط العلوي: رجوع (يمين) ثم العنوان، وعنصر اختياري في الطرف الآخر.
class ScreenTopBar extends StatelessWidget {
  const ScreenTopBar({
    super.key,
    required this.title,
    this.subtitle,
    required this.onBack,
    this.end,
  });
```

ماذا يفعل؟ تعليق وصنف `ScreenTopBar` وباني: العنوان والرجوع مطلوبان، والعنوان الفرعي والعنصر الأخير اختياريان.

لماذا بهذه الطريقة؟ يوفّر نفس الشريط لشاشات 02 و26 وغيرها.

الأسطر 50–53

```
  final String title;
  final String? subtitle;
  final VoidCallback onBack;
  final Widget? end;
```

ماذا يفعل؟ الحقول.

لماذا بهذه الطريقة؟ غير قابلة للتغيير.

الأسطر 55–59

```
  @override
  Widget build(BuildContext context) {
    return Padding(
      padding: const EdgeInsetsDirectional.fromSTEB(20, 16, 20, 4),
      child: Row(
```

ماذا يفعل؟ `build` وحشوة الشريط 20/16/20/4 و `Row` .

لماذا بهذه الطريقة؟ أول عنصر على اليمين: زر الرجوع.

الأسطر 60–62

```
        spacing: 12,
        children: [
          BackTile(onTap: onBack),
```

ماذا يفعل؟ مسافة 12 وزر الرجوع أولًا.

لماذا بهذه الطريقة؟ بحسب ترتيب Figma في RTL.

الأسطر 63–72

```
63            Expanded(
64              child: Column(
65                crossAxisAlignment: CrossAxisAlignment.start,
66                children: [
67                  if (subtitle != null)
68                    Text(subtitle!, style: AppTextStyles.caption13),
69                  Text(title, style: AppTextStyles.heading22),
70                ],
71              ),
72            ),
```

ماذا يفعل؟ `Expanded` فيه عمود: العنوان الفرعي إن وُجد ثم العنوان.

لماذا بهذه الطريقة؟ `Expanded` يملأ ما بين الزر والعنصر الأخير. المحاذاة `start` = اليمين في RTL.

السطر 73

```
73            ?end,
```

ماذا يفعل؟ عنصر اختياري `?end` في الطرف الآخر.

لماذا بهذه الطريقة؟ الصيغة `?x` تضيف العنصر إلى القائمة فقط إن لم يكن `null` (Dart 3.8).

الأسطر 74–78

```
74          ],
75        ),
76      );
77    }
78  }
```

ماذا يفعل؟ إغلاق الأقواس.

لماذا بهذه الطريقة؟ نهاية الملف.

lib/core/widgets/rizma_switch.dart — مفتاح التشغيل

lib/core/widgets/rizma_switch.dart

الأسطر 1–5

```
1   import 'package:flutter/material.dart';
2
3   import '../theme/app_colors.dart';
4
5   /// مفتاح التشغيل بحسب التصميم (32×52). المقبض يبدأ من اليمين وهو مطفأ (RTL).
```

ماذا يفعل؟ استيراد Material والألوان وتعليق الصنف.

لماذا بهذه الطريقة؟ التصميم 52×32؛ والمقبض يبدأ من اليمين وهو مطفأ (RTL).

الأسطر 6–12

```
6   class RizmaSwitch extends StatelessWidget {
7     const RizmaSwitch({
8       super.key,
9       required this.value,
10      required this.onChanged,
11      required this.semanticsLabel,
12    });
```

ماذا يفعل؟ صنف `RizmaSwitch` وباني بثلاثة معاملات مطلوبة.

لماذا بهذه الطريقة؟ القيمة والدالة ووصف قارئ الشاشة.

الأسطر 14–16

```
14    final bool value;
15    final ValueChanged<bool> onChanged;
16    final String semanticsLabel;
```

ماذا يفعل؟ الحقول.

لماذا بهذه الطريقة؟ الودجت بلا حالة؛ المالك يحتفظ بالقيمة.

الأسطر 18–19

```
18    @override
19    Widget build(BuildContext context) {
```

ماذا يفعل؟ `build` .

لماذا بهذه الطريقة؟ يعيد المفتاح.

الأسطر 20–24

```
20      return Semantics(
21        toggled: value,
22        label: semanticsLabel,
23        excludeSemantics: true,
24        onTap: () => onChanged(!value),
```

ماذا يفعل؟ `Semantics` تعلن مفتاحًا بحالته (`toggled`) واسمه وتستجيب للمس.

لماذا بهذه الطريقة؟ ضروري لقارئ الشاشة لأن المفتاح مرسوم يدويًا.

الأسطر 25–27

```
25        child: GestureDetector(
26          onTap: () => onChanged(!value),
27          behavior: HitTestBehavior.opaque,
```

ماذا يفعل؟ `GestureDetector` يقلب القيمة عند الضغط.

لماذا بهذه الطريقة؟ `HitTestBehavior.opaque` يجعل كل المساحة قابلة للضغط حتى الشفافة.

الأسطر 28–32

```
28          child: AnimatedContainer(
29            duration: const Duration(milliseconds: 150),
30            width: 52,
31            height: 32,
32            padding: const EdgeInsets.all(4),
```

ماذا يفعل؟ `AnimatedContainer` بحجم 52×32 وحشوة 4.

لماذا بهذه الطريقة؟ الانتقال ناعم 150 مللي ثانية.

الأسطر 33–36

```
33            decoration: BoxDecoration(
34              color: value ? AppColors.teal : AppColors.disabled,
35              borderRadius: BorderRadius.circular(16),
36            ),
```

ماذا يفعل؟ لون المسار: teal إن مفعّل وإلا `disabled`، وزوايا 16.

لماذا بهذه الطريقة؟ مطابق لـ Figma.

الأسطر 37–41

```
37            child: AnimatedAlign(
38              duration: const Duration(milliseconds: 150),
39              alignment: value
40                  ? AlignmentDirectional.centerEnd
41                  : AlignmentDirectional.centerStart,
```

ماذا يفعل؟ `AnimatedAlign`: المقبض عند `centerEnd` إن مفعّل وإلا `centerStart`.

لماذا بهذه الطريقة؟ بصيغة `Directional` فيبدأ من اليمين وهو مطفأ في RTL.

الأسطر 42–48

```
42              child: const DecoratedBox(
43                decoration: BoxDecoration(
44                  color: AppColors.card,
45                  shape: BoxShape.circle,
46                ),
47                child: SizedBox(width: 24, height: 24),
48              ),
```

ماذا يفعل؟ المقبض دائرة بيضاء 24×24.

لماذا بهذه الطريقة؟ من Figma.

الأسطر 49–54

```
          ),
        ),
      ),
    );
  }
}
```

ماذا يفعل؟ إغلاق الأقواس.

لماذا بهذه الطريقة؟ نهاية الملف.

lib/core/widgets/rizma_card.dart — البطاقة

lib/core/widgets/rizma_card.dart

الأسطر 1–5

```
import 'package:flutter/material.dart';

import '../theme/app_colors.dart';

/// بطاقة بيضاء بحد رفيع وزوايا 20 (CARD في التصميم).
```

ماذا يفعل؟ استيراد Material والألوان وتعليق الصنف.

لماذا بهذه الطريقة؟ بطاقة بيضاء بحد رفيع وزوايا 20.

الأسطر 6–11

```
class RizmaCard extends StatelessWidget {
  const RizmaCard({
    super.key,
    required this.child,
    this.padding = const EdgeInsets.all(16),
  });
```

ماذا يفعل؟ الباني: `child` مطلوب، والحشوة الافتراضية 16.

لماذا بهذه الطريقة؟ القيمة الأكثر استعمالًا في التصميم؛ الشاشات تمرّر غيرها عند الحاجة.

الأسطر 13–14

```
  final Widget child;
  final EdgeInsetsGeometry padding;
```

ماذا يفعل؟ الحقلان: المحتوى والحشوة (`EdgeInsetsGeometry`).

لماذا بهذه الطريقة؟ النوع العام يقبل `Directional`.

الأسطر 16–17

```
16    @override
17    Widget build(BuildContext context) {
```

ماذا يفعل؟ `build` .

لماذا بهذه الطريقة؟ يعيد البطاقة.

الأسطر 18–23

```
18      return DecoratedBox(
19        decoration: BoxDecoration(
20          color: AppColors.card,
21          borderRadius: BorderRadius.circular(20),
22          border: Border.all(color: AppColors.line),
23        ),
```

ماذا يفعل؟ `DecoratedBox` بخلفية بيضاء وزوايا 20 وحد `line` .

لماذا بهذه الطريقة؟ أخف من `Container` حين لا نحتاج حجمًا.

الأسطر 24–27

```
24        child: Padding(padding: padding, child: child),
25      );
26    }
27  }
```

ماذا يفعل؟ `Padding` حول المحتوى ثم إغلاق الأقواس.

لماذا بهذه الطريقة؟ الحشوة داخل الإطار المزخرف.

lib/core/widgets/dashed_border.dart — الحد المنقّط

lib/core/widgets/dashed_border.dart

الأسطر 1–4

```
1   import 'package:flutter/material.dart';
2
3   /// صندوق بحد منقّط وزوايا مدوّرة (الصناديق الفارغة في التصميم).
4   class DashedBorderBox extends StatelessWidget {
```

ماذا يفعل؟ استيراد Material وتعليق وصنف `DashedBorderBox` .

لماذا بهذه الطريقة؟ تُستعمل في «طالب جديد» وصندوق المراجعة الفارغ.

الأسطر 5–14

```
5     const DashedBorderBox({
6       super.key,
7       required this.child,
8       required this.color,
9       this.radius = 18,
10      this.strokeWidth = 2,
11      this.dash = 6,
12      this.gap = 5,
13      this.padding = EdgeInsets.zero,
14    });
```

ماذا يفعل؟ الباني: المحتوى واللون مطلوبان، والباقي اختياري (زوايا 18، سُمك 2، شرطة 6، فراغ 5، حشوة صفر).

لماذا بهذه الطريقة؟ قابل للتخصيص: «طالب جديد» يستعمل شرطة 8 وفراغ 6.

الأسطر 16–22

```
16    final Widget child;
17    final Color color;
18    final double radius;
19    final double strokeWidth;
20    final double dash;
21    final double gap;
22    final EdgeInsetsGeometry padding;
```

ماذا يفعل؟ الحقول.

لماذا بهذه الطريقة؟ تُمرَّر للرسّام.

الأسطر 24–26

```
24    @override
25    Widget build(BuildContext context) {
26      return CustomPaint(
```

ماذا يفعل؟ `build` يعيد `CustomPaint`.

لماذا بهذه الطريقة؟ لا يوجد في Flutter حد منقّط جاهز لمستطيل مدوّر، فنرسمه.

الأسطر 27–33

```
27        painter: _DashedRRectPainter(
28          color: color,
29          radius: radius,
30          strokeWidth: strokeWidth,
31          dash: dash,
32          gap: gap,
33        ),
```

ماذا يفعل؟ نمرّر للرسّام اللون والزوايا والسُمك والشرطة والفراغ.

لماذا بهذه الطريقة؟ رسّام خاص بالملف.

الأسطر 34–37

```
        child: Padding(padding: padding, child: child),
      );
    }
  }
```

ماذا يفعل؟ الابن داخل `Padding` ثم إغلاق الودجت.

لماذا بهذه الطريقة؟ المحتوى يظهر فوق الحدّ المرسوم.

الأسطر 39–46

```
class _DashedRRectPainter extends CustomPainter {
  _DashedRRectPainter({
    required this.color,
    required this.radius,
    required this.strokeWidth,
    required this.dash,
    required this.gap,
  });
```

ماذا يفعل؟ الصنف الخاص `_DashedRRectPainter` وبانيه.

لماذا بهذه الطريقة؟ `CustomPainter` هو الصنف الذي يرسم على اللوحة.

الأسطر 48–52

```
  final Color color;
  final double radius;
  final double strokeWidth;
  final double dash;
  final double gap;
```

ماذا يفعل؟ الحقول.

لماذا بهذه الطريقة؟ غير قابلة للتغيير.

الأسطر 54–55

```
  @override
  void paint(Canvas canvas, Size size) {
```

ماذا يفعل؟ دالة `paint(canvas, size)`.

لماذا بهذه الطريقة؟ تُستدعى عند الرسم.

الأسطر 62–56

```
56      final inset = strokeWidth / 2;
57      final rect = Rect.fromLTWH(
58        inset,
59        inset,
60        size.width - strokeWidth,
61        size.height - strokeWidth,
62      );
```

ماذا يفعل؟ نحسب مستطيلًا مزاحًا بنصف سُمك الخط من كل جهة.

لماذا بهذه الطريقة؟ كي لا يُقصّ نصف الخط عند حواف الودجت.

الأسطر 64–63

```
63      final path = Path()
64        ..addRRect(RRect.fromRectAndRadius(rect, Radius.circular(radius)));
```

ماذا يفعل؟ نبني مسارًا بمستطيل مدوّر.

لماذا بهذه الطريقة؟ `RRect` = مستطيل بزوايا مدوّرة.

الأسطر 68–65

```
65      final paint = Paint()
66        ..color = color
67        ..style = PaintingStyle.stroke
68        ..strokeWidth = strokeWidth;
```

ماذا يفعل؟ فرشاة بلون الحد ونمط «خط» وسُمك محدد.

لماذا بهذه الطريقة؟ `PaintingStyle.stroke` يرسم الحدود لا التعبئة.

الأسطر 77–70

```
70      for (final metric in path.computeMetrics()) {
71        var distance = 0.0;
72        while (distance < metric.length) {
73          final end = (distance + dash).clamp(0, metric.length).toDouble();
74          canvas.drawPath(metric.extractPath(distance, end), paint);
75          distance += dash + gap;
76        }
77      }
```

ماذا يفعل؟ نمرّ على كل مقطع من المسار ونرسم شرطات بطول `dash` يفصل بينها `gap`.

لماذا بهذه الطريقة؟ `computeMetrics` تعطينا طول المسار لنقتطع منه شرطات متتالية. `clamp` يمنع تجاوز نهاية المسار.

السطر 78

```
78    }
```

ماذا يفعل؟ إغلاق `paint`.

لماذا بهذه الطريقة؟ نهاية الدالة.

الأسطر 80–87

```
  @override
  bool shouldRepaint(_DashedRRectPainter old) =>
      old.color != color ||
      old.radius != radius ||
      old.strokeWidth != strokeWidth ||
      old.dash != dash ||
      old.gap != gap;
}
```

ماذا يفعل؟ `shouldRepaint`: نعيد الرسم فقط إن تغيّرت إحدى القيم، ثم إغلاق الصنف.

لماذا بهذه الطريقة؟ تحسين أداء: لا نرسم من جديد بلا داعٍ.

lib/core/widgets/pin_pad.dart — نقاط الرمز السري ولوحة الأرقام

lib/core/widgets/pin_pad.dart

الأسطر 1–4

```
import 'package:flutter/material.dart';

import '../strings/ar.dart';
import '../theme/app_colors.dart';
```

ماذا يفعل؟ استيراد Material والنصوص والألوان.

لماذا بهذه الطريقة؟ ما تحتاجه النقاط واللوحة.

الأسطر 6–13

```
/// نقاط الرمز السري: المملوء منها = أرقام أُدخلت.
class PinDots extends StatelessWidget {
  const PinDots({
    super.key,
    required this.length,
    required this.filled,
    this.error = false,
  });
```

ماذا يفعل؟ تعليق وصنف `PinDots` وباني: `length` و`filled` مطلوبان و`error` افتراضيه false.

لماذا بهذه الطريقة؟ النقاط تعرض عدد الأرقام المُدخلة دون كشفها.

الأسطر 15–17

```
  final int length;
  final int filled;
  final bool error;
```

ماذا يفعل؟ الحقول.

لماذا بهذه الطريقة؟ الطول والمُدخَل والخطأ.

الأسطر 19–20

```
19    @override
20    Widget build(BuildContext context) {
```

ماذا يفعل؟ `build` .

لماذا بهذه الطريقة؟ يعيد النقاط.

الأسطر 21–23

```
21      return Semantics(
22        label: ArStrings.pinDotsSemantics(filled, length),
23        excludeSemantics: true,
```

ماذا يفعل؟ `Semantics` تقرأ «أُدخل N من 4 أرقام».

لماذا بهذه الطريقة؟ لا نكشف الأرقام لقارئ الشاشة.

الأسطر 24–29

```
24        child: Padding(
25          padding: const EdgeInsets.symmetric(vertical: 10),
26          child: Row(
27            mainAxisSize: MainAxisSize.min,
28            spacing: 18,
29            children: [
```

ماذا يفعل؟ حشوة عمودية 10 و`Row` بحجم أدنى ومسافة 18.

لماذا بهذه الطريقة؟ مسافة التصميم بين النقاط.

الأسطر 30–32

```
30              for (var i = 0; i < length; i++)
31                AnimatedContainer(
32                  duration: const Duration(milliseconds: 120),
```

ماذا يفعل؟ حلقة تبني نقطة لكل رقم بـ `AnimatedContainer` (120 مللي ثانية).

لماذا بهذه الطريقة؟ الامتلاء يظهر بحركة قصيرة ناعمة.

الأسطر 33–36

```
33                  width: 20,
34                  height: 20,
35                  decoration: BoxDecoration(
36                    shape: BoxShape.circle,
```

ماذا يفعل؟ حجم 20×20 وشكل دائري.

لماذا بهذه الطريقة؟ من Figma.

الأسطر 37–39

```
                    color: i < filled
                        ? (error ? AppColors.coral : AppColors.teal)
                        : Colors.transparent,
```

ماذا يفعل؟ النقطة المملوءة: teal (أو مرجاني عند الخطأ)، وغير المملوءة شفافة.

لماذا بهذه الطريقة؟ اللون المرجاني يدل على إدخال خاطئ.

الأسطر 40–47

```
                    border: i < filled
                        ? null
                        : Border.all(
                            color: error ? AppColors.coral : AppColors.disabled,
                            width: 2,
                          ),
                  ),
                ),
```

ماذا يفعل؟ الحد: لا حد للمملوءة، وحد 2 رمادي (أو مرجاني) للفارغة.

لماذا بهذه الطريقة؟ الفارغة دائرة بحد، والمملوءة دائرة مصمتة.

الأسطر 48–53

```
            ],
          ),
        ),
      );
    }
}
```

ماذا يفعل؟ إغلاق الأقواس.

لماذا بهذه الطريقة؟ نهاية `PinDots`.

الأسطر 55–57

```
/// لوحة أرقام الرمز السري. مبنية LTR (فراغ 0 حذف / 9 8 7 / 6 5 4 / 3 2 1) عمدًا
/// كما في التصميم، حتى في واجهة RTL.
class PinPad extends StatelessWidget {
```

ماذا يفعل؟ تعليق وصنف `PinPad`.

لماذا بهذه الطريقة؟ اللوحة مبنية LTR عمدًا: 1 2 3 / 4 5 6 / 7 8 9 / فراغ 0 حذف كما في التصميم.

الأسطر 58–63

```
  const PinPad({
    super.key,
    required this.onDigit,
    required this.onDelete,
    this.enabled = true,
  });
```

ماذا يفعل؟ الباني: دالة الرقم ودالة الحذف مطلوبتان و`enabled` افتراضيه true.

لماذا بهذه الطريقة؟ `enabled: false` أثناء القفل.

الأسطر 65–67

```
  final ValueChanged<String> onDigit;
  final VoidCallback onDelete;
  final bool enabled;
```

ماذا يفعل؟ الحقول.

لماذا بهذه الطريقة؟ الدوال والحالة.

الأسطر 69–74

```
  static const _rows = [
    ['1', '2', '3'],
    ['4', '5', '6'],
    ['7', '8', '9'],
    ['', '0', 'del'],
  ];
```

ماذا يفعل؟ ثابت `_rows`: أرقام اللوحة؛ الفراغ نص فارغ والحذف `del`.

لماذا بهذه الطريقة؟ نبني اللوحة من بيانات لا من 12 ودجت يدويًا.

الأسطر 76–77

```
  @override
  Widget build(BuildContext context) {
```

ماذا يفعل؟ `build` (توقيع الدالة).

لماذا بهذه الطريقة؟ تعيد اللوحة كلها.

الأسطر 78–80

```
    // FittedBox: تتقلّص اللوحة بدل أن تفيض (عرض اللوحة 276) على شاشة أضيق من.
    return FittedBox(
      fit: BoxFit.scaleDown,
```

ماذا يفعل؟ تعليق ثم `FittedBox(fit: scaleDown)` يلفّ اللوحة.

لماذا بهذه الطريقة؟ عرض اللوحة الطبيعي 276 (3 مفاتيح × 80 + مسافتان 18). في ورقة سفلية على شاشة 320 يبقى 272 فقط فكانت الصفوف تفيض بـ 4 بكسلات (اكتشفه `layout_stress_test`). `scaleDown` يصغّرها قليلًا عند الضيق ولا يكبّرها أبدًا.

الأسطر 81–82

```
81        child: Directionality(
82          textDirection: TextDirection.ltr,
```

ماذا يفعل؟ `Directionality(ltr)` داخل `FittedBox`.

لماذا بهذه الطريقة؟ يُبقي ترتيب الأرقام عالميًا (1 يسارًا) مهما كان اتجاه الشاشة.

الأسطر 83–86

```
83          child: Column(
84            mainAxisSize: MainAxisSize.min,
85            spacing: 12,
86            children: [
```

ماذا يفعل؟ `Column` بحجم أدنى ومسافة 12 بين الصفوف وبداية `children`.

لماذا بهذه الطريقة؟ مسافة Figma.

الأسطر 87–92

```
87              for (final row in _rows)
88                Row(
89                  mainAxisSize: MainAxisSize.min,
90                  spacing: 18,
91                  children: [for (final key in row) _buildKey(key)],
92                ),
```

ماذا يفعل؟ لكل صف `Row` بمسافة 18 يضم مفاتيحه.

لماذا بهذه الطريقة؟ نبني الصفوف من بيانات `_rows`.

الأسطر 93–97

```
93            ],
94          ),
95        ),
96      );
97    }
```

ماذا يفعل؟ إغلاق `Column` و`Directionality` و`FittedBox` و`build`.

لماذا بهذه الطريقة؟ نهاية الدالة.

الأسطر 99–100

```
99    Widget _buildKey(String key) {
100     if (key.isEmpty) return const SizedBox(width: 80, height: 64);
```

ماذا يفعل؟ `_buildKey`: المفتاح الفارغ مساحة 80×64 بلا رسم.

لماذا بهذه الطريقة؟ يحفظ مكان الخانة الفارغة.

الأسطر 101–112

```
    if (key == 'del') {
      return _PinKey(
        semanticsLabel: ArStrings.pinDelete,
        onTap: enabled ? onDelete : null,
        filled: false,
        child: const Icon(
          Icons.backspace_outlined,
          size: 26,
          color: AppColors.ink,
        ),
      );
    }
```

ماذا يفعل؟ مفتاح الحذف: اسم «حذف» وأيقونة `backspace` بلا خلفية؛ ويتعطل إن كانت اللوحة معطّلة.

لماذا بهذه الطريقة؟ `onTap: null` يعطّله.

الأسطر 113–116

```
    return _PinKey(
      semanticsLabel: key,
      onTap: enabled ? () => onDigit(key) : null,
      filled: true,
```

ماذا يفعل؟ مفتاح رقم: اسمه الرقم نفسه وينادي `onDigit` وخلفيته بيضاء.

لماذا بهذه الطريقة؟ قارئ الشاشة يقرأ الرقم.

الأسطر 117–128

```
      child: Text(
        key,
        style: const TextStyle(
          fontFamily: 'Cairo',
          fontSize: 28,
          fontWeight: FontWeight.w700,
          height: 1.2,
          color: AppColors.ink,
        ),
      ),
    );
  }
```

ماذا يفعل؟ نص الرقم بنمط يدوي: Cairo حجم 28 وزن 700.

لماذا بهذه الطريقة؟ نمط خاص بالمفاتيح ولا يتكرر في مكان آخر فلا نضيفه لـ `AppTextStyles`.

السطر 129

```
}
```

ماذا يفعل؟ إغلاق `PinPad`.

لماذا بهذه الطريقة؟ نهاية الصنف.

الأسطر 131–137

```
131 class _PinKey extends StatelessWidget {
132   const _PinKey({
133     required this.semanticsLabel,
134     required this.onTap,
135     required this.filled,
136     required this.child,
137   });
```

ماذا يفعل؟ صنف خاص `_PinKey` وباني بأربعة معاملات.

لماذا بهذه الطريقة؟ مفتاح واحد من اللوحة.

الأسطر 139–142

```
139   final String semanticsLabel;
140   final VoidCallback? onTap;
141   final bool filled;
142   final Widget child;
```

ماذا يفعل؟ الحقول.

لماذا بهذه الطريقة؟ الاسم والدالة وهل له خلفية والمحتوى.

الأسطر 144–150

```
144   @override
145   Widget build(BuildContext context) {
146     return Semantics(
147       button: true,
148       enabled: onTap != null,
149       label: semanticsLabel,
150       excludeSemantics: true,
```

ماذا يفعل؟ `build` و`Semantics` لزر بحالة تفعيله واسمه.

لماذا بهذه الطريقة؟ يعمل قارئ الشاشة بلا الحاجة لقراءة محتوى المفتاح.

الأسطر 151–158

```
151       child: Material(
152         color: filled ? AppColors.card : Colors.transparent,
153         shape: StadiumBorder(
154           side: filled
155               ? const BorderSide(color: AppColors.line)
156               : BorderSide.none,
157         ),
158         clipBehavior: Clip.antiAlias,
```

ماذا يفعل؟ `Material` بخلفية بيضاء أو شفافة وشكل «كبسولة» (`StadiumBorder`) وحد `line` للأرقام فقط، وقصّ.

لماذا بهذه الطريقة؟ مفتاح الحذف بلا حد ولا خلفية كما في Figma.

الأسطر 159–166

```
        child: InkWell(
          onTap: onTap,
          child: SizedBox(width: 80, height: 64, child: Center(child: child)),
        ),
      ),
    );
  }
}
```

ماذا يفعل؟ `InkWell` وصندوق 80×64 بمحتوى في المنتصف، ثم إغلاق الأقواس.

لماذا بهذه الطريقة؟ مقاس المفتاح من التصميم.